Package 'JumpForests'

October 8, 2026

Type Package

Title Adaptive Conditional Aalen-Johansen Estimation

Version 1.0

Date 2026-03-05

Maintainer Martin Bladt <martinbladt@math.ku.dk>

Description Provides random forest versions of the Nelson-Aalen and Aalen-Johansen estimators. The methods are based on Bladt & Lemvig (2025+), in preparation.

License GPL-2 | file LICENSE

Imports Rcpp,
stats,
utils

LinkingTo Rcpp

SystemRequirements C++17, OpenMP

Encoding UTF-8

Config/roxygen2/version 8.1.0

Contents

JumpForests-package 2
jfforest 2
jfforest.error 5
jfforest.predict 6
jfforest.vimp 8
jftree 9
jftree.error 11
jftree.predict 13
occupation_prob 14
predict.jf_forest 15
predict.jf_tree 16
print.jf_forest 17
print.jf_tree 18
print.jf_vimp 18

print_forest 19
print_tree 19
transition_prob 20

Index **21**

`JumpForests-package` *JumpForests: Adaptive Conditional Aalen-Johansen Estimation*

Description

Provides random forest versions of the Nelson-Aalen and Aalen-Johansen estimators. The methods are based on Bladt & Lemvig (2025+), in preparation.

Author(s)

Maintainer: Martin Bladt <martinbladt@math.ku.dk>

Authors:

- Martin Bladt <martinbladt@math.ku.dk>
- Rasmus Lemvig <rfl@math.ku.dk>

`jfforest` *Fit a JumpForests forest*

Description

Fits a random forest for regression, classification, survival, or multi-state data.

Usage

```
jfforest(
  formula,
  data,
  feature_data = NULL,
  splitrule = NULL,
  mtry = NULL,
  min_node_size = NULL,
  max_depth = NULL,
  nsplits = 10,
  ntrees = NULL,
  honest = FALSE,
  swr = FALSE,
  sample_rate = NULL,
  double_bootstrap = FALSE,
  seed = NULL,
```

```
  nworkers = 0,
  save_predictions = TRUE,
  num_event_times = 0,
  state_weights = NULL,
  fh_weights_a = NULL,
  fh_weights_b = NULL,
  splitrule_par = NULL
)
```

Arguments

formula	Model formula: `y ~ x1 + x2` for regression or classification, `Surv(time, status) ~ x1 + x2` for survival (`status = 1` for an event, `0` for censoring), or `MM ~ x1 + x2` for multi-state models. A numeric response selects regression; a factor, character, or logical response selects classification. Convert numeric class labels to a factor to fit a classification forest. Numeric features are treated as continuous; factor, character, and logical features are treated as categorical.
data	Training data. A data.frame including both responses and features for regression, classification and survival. For multi-state models, a list of lists (one for each observation), each containing two vectors, `"times"` (observed jump times) and `"states"` (the state occupied at the corresponding time). The states must be encoded as integers. Repetition of the final state indicates censoring.
feature_data	Feature data as a data.frame. Only required for multi-state models.
splitrule	Splitting rule. `NULL` uses the model-specific defaults below. For regression, choose `"mse"` (default), `"variance"` (an alias for `"mse"`), `"mae"`, `"binomial"`, `"negativebinomial"`, `"poisson"`, `"gamma"`, `"inversegaussian"`, `"tweedie"`, or `"huber"`. Binomial responses must lie in `[0, 1]`; negative-binomial and Poisson responses must be non-negative integers; Gamma and inverse-Gaussian responses must be positive. Tweedie support follows its power: real for $\xi \leq 0$, non-negative counts at $\xi = 1$, non-negative for $1 < \xi < 2$, and positive for $\xi \geq 2$. For classification, choose `"gini"` (default), `"entropy"`, `"misc"`, `"twoing"` or `"hellinger"`. For survival, choose `"logrank"` (default), `"approxlogrank"` or `"conserve"`. For multi-state models, choose `"logrank"` (default), `"gehan"`, `"taroneware"`, `"approxlogrank"`, `"conserve"`, `"petoprentice"` or `"flemingharrington"`. Choosing `"flemingharrington"` allows for specifying the weights, see the options `"fh_weights_a"` and `"fh_weights_b"`.
mtry	Number of randomly selected candidate features at each split. `NULL` uses `ceiling(sqrt(p))`, where `p` is the number of features.
min_node_size	Minimum number of observations required in each daughter node for a split. `NULL` uses 5 for regression, 1 for classification, 15 for survival, and 20 for multi-state models.
max_depth	Maximum depth allowed for each tree, where the root has depth zero. `NULL` (the default) imposes no depth limit.
nsplits	Number of randomly sampled candidate thresholds per continuous feature, or candidate partitions per categorical feature. Defaults to 10; fewer candidates are used if fewer distinct splits exist.

`ntrees` Number of trees. `NULL` uses 1000 for regression and classification, and 500 for survival and multi-state models.

`honest` Whether to use separate sets for choosing splits and estimating leaf predictions, each approximately half of the tree's sample. Defaults to `FALSE`, which uses the same observations for both tasks. See `double_bootstrap` for how these sets are sampled.

`swr` Whether to sample with replacement. Defaults to `FALSE`.

`sample_rate` Fraction of observations sampled for each tree, in `(0, 1]`. `NULL` uses 0.7 without replacement (`swr = FALSE`) and 1 with replacement (`swr = TRUE`). With `honest = TRUE` and `double_bootstrap = TRUE`, the rate applies to each half separately.

`double_bootstrap`
Only relevant when `honest = TRUE`. If `TRUE`, first partition the training observations into disjoint halves, then sample separately within each half for splitting and leaf estimation. If `FALSE` (the default), sample first and then partition the sample into halves. With replacement, the latter can put copies of the same observation in both halves.

`seed` Random seed. `NULL` draws a seed using R's current random-number state; supply a seed for reproducible fitting.

`nworkers` Number of worker threads. The default, 0, uses all hardware threads reported by the system.

`save_predictions`
Whether to compute and store ordinary training predictions, OOB predictions, and errors during fitting for survival and multi-state forests. Defaults to `TRUE`; `FALSE` defers these calculations until needed. Regression and classification currently always store predictions and errors, regardless of this flag.

`num_event_times`
Maximum number of times in the prediction grid for survival and multi-state models. The default, 0, retains the full grid. A positive value coarsens the grid when necessary; the resulting times are stored in `unique.event.times`.

`state_weights` Numeric vector of finite non-negative weights, one per state in the order `1, ..., num_states`, for multi-state error calculations. Supplied weights are used directly for all scores and are not normalised. `NULL` uses `1 / num_states` per state for Brier and spherical scores and 1 per state for Kullback-Leibler loss. These weights affect error calculations, not splitting or prediction.

`fh_weights_a` Optional finite non-negative Fleming–Harrington a-exponents, one for each state `1, ..., num_states`. `NULL` uses one for every state. Only valid for multi-state forests with `splitrule = "flemingharrington"`. A transition from state `s` is weighted by $p_s(t)^{a_s}(1-p_s(t))^{b_s}$, where $p_s(t)$ is the estimated occupation probability of state `s` in the parent node.

`fh_weights_b` Optional finite non-negative Fleming–Harrington b-exponents, with the same ordering and default as `fh_weights_a`.

`splitrule_par` Numeric scalar for parameterised regression rules: negative-binomial size $k > 0$ (default 1), Tweedie power $\xi \leq 0$ or $\xi \geq 1$, or Huber threshold $\delta > 0$. It is required for Tweedie and Huber and must be `NULL` for every other rule and model type. The default is `NULL`.

Value

A fitted forest object as a list with class `jf_forest`. Use `print()` for a summary and `predict()` for predictions. When saved, `predictions` contains ordinary training predictions using all trees, and `oob.predictions` contains OOB predictions. Classification also stores the corresponding probability matrices in `predictions.prob` and `oob.predictions.prob`. For survival and multi-state forests, `censoring.oob` contains OOB estimates of the censoring survival probability obtained by averaging leaf Kaplan-Meier estimators, with one row per observation and one column per time in `unique.event.times`. OOB estimates require OOB trees for the observation. With `save_predictions = FALSE`, survival and multi-state prediction and error fields are initially `NULL`.

`jfforest.error` *Compute forest prediction error*

Description

Compute forest prediction error

Usage

```
jfforest.error(
  forest_list,
  new_data = NULL,
  jump_data = NULL,
  state_weights = NULL
)
```

Arguments

`forest_list`	A fitted forest object from `jfforest()`.
`new_data`	Optional evaluation data.frame. For regression, classification, and survival, include all fitted feature and response columns. For multi-state models, include the features here and supply responses in `jump_data`. Variable types and categorical levels must match the training data; responses must be fully observed apart from the model's censoring representation. `NULL` requests OOB errors.
`jump_data`	Multi-state response paths in the same format as `jfforest()`'s `data`, with one path per row of `new_data` in the same order. Required only for evaluating multi-state models on new data; defaults to `NULL`.
`state_weights`	Multi-state error weights as described in `jfforest()`. Without new data, `NULL` returns stored OOB errors when available, retaining the weights used during fitting. Supplying weights recomputes the OOB errors with those weights. With new data, or when OOB errors must first be computed, `NULL` uses the loss-specific defaults.

Details

Without new_data, this function returns OOB errors, computing deferred survival or multi-state errors when needed. With new_data, predictions use all trees and errors are evaluated on those observations. Survival and multi-state probability scores account for censoring using inverse censoring probability weights. Training errors use OOB censoring estimates; new-data errors use censoring estimates from all trees.

Value

A named list of error metrics. Smaller losses are better; larger R2 is better.

- Regression: mse.error for OOB evaluation, or error.mse with new data, is mean squared error. R2 is $1 - \mathrm{MSE}/\mathrm{Var}(y)$, using the empirical variance with divisor n on the evaluated responses; it can be negative.
- Classification: BS.error is the squared class-probability error averaged over observations and classes. normalised.BS.error divides this by the score for uniform class probabilities, so 1 is the uniform baseline. misclassification.error.total is the overall proportion of wrong labels; misclassification.error gives this proportion within each observed class (or NA if that class is absent). confusion.matrix contains counts, with observed classes in rows and predicted classes in columns.
- Survival: C.error is 1 - C-index, so smaller values mean better concordance. IBS.error is the integrated Brier score for survival probabilities; KL.error for OOB evaluation, or IKL.error with new data, is the integrated Kullback-Leibler (logarithmic) loss. The time-normalised fields are normalised.IBS.error and normalised.KL.error (OOB) or normalised.IKL.error (new data).
- Multi-state: IBS.error, IKL.error, and IS.error are the integrated Brier, Kullback-Leibler, and spherical losses for state occupation probabilities, with state weights as specified above. The corresponding fields are normalised.IBS.error, normalised.IKL.error, and normalised.IS.error.

Integrated scores use trapezoidal integration on the fitted event-time grid. Their normalised.* versions divide by the grid's final time, giving an average loss over time; normalisation does not guarantee a value between zero and one. This differs from the uniform-probability normalisation used for classification Brier scores.

---

jfforest.predict *Predict from a fitted forest*

Description

Predict from a fitted forest

Usage

```
jfforest.predict(
  forest_list,
  new_data = NULL,
```

```
  compute_censoring = FALSE,
  compute_initial = FALSE
)
```

Arguments

forest_list	A fitted forest object from jfforest().
new_data	Optional data.frame containing all fitted feature names, with the same variable types as the training data. Response columns are not needed; extra columns are ignored. Non-missing categorical values must occur in the training levels. Missing feature values are sent to the right daughter at each split on that feature. NULL returns ordinary training predictions using all trees. Stored OOB predictions are in forest_list$oob.predictions.
compute_censoring	Should the censoring survival distribution also be estimated? For multi-state forests, returns out-of-bag estimates for training data and ordinary estimates for new_data. For survival forests, this applies when new_data is supplied. Defaults to FALSE; only valid for survival and multi-state forests. Estimates are averages of leaf Kaplan-Meier censoring survival probabilities.
compute_initial	Should predicted initial distributions also be returned? For multi-state forests with deferred predictions, this also controls whether they are computed during the prediction call. Defaults to FALSE; only valid for multi-state forests.

Details

With new_data = NULL, returned predictions (and multi-state initial distributions when requested) use all trees. OOB predictions are stored separately in forest_list$oob.predictions; training censoring estimates use OOB trees. With new_data, all returned estimates use all trees. Training censoring estimates require OOB trees for the observation. With deferred predictions, this call computes the requested estimates; use the returned object to access them.

Value

Regression: A numeric vector with one predicted response per training observation. With new_data, a numeric matrix with one column and one row per observation.

Classification: A numeric vector of predicted class codes for training observations. With new_data, a numeric matrix whose first column, prediction, contains the class codes and whose remaining columns contain class probabilities, named by forest_list$class.levels. Class codes index these class levels. Training class probabilities are stored separately in forest_list$predictions.prob.

Survival: A matrix of estimated cumulative hazards (Nelson-Aalen estimators), with one row per observation and one column per time in forest_list$unique.event.times. With new_data and compute_censoring = TRUE, a list containing this matrix in predictions and censoring Kaplan-Meier estimates on the same grid in censoring. Without new_data, the returned object is the cumulative hazard matrix; stored out-of-bag censoring estimates, when available, can be accessed through forest_list$censoring.oob.

Multi-state: Predictions are a list of subjects when both optional estimates are disabled; access one subject with `result[[subject]]`. Otherwise, `result$predictions` contains the subject predictions. With `compute_initial = TRUE`, the result also has `init`. With `compute_censoring = TRUE`, it has `censoring`, a subject-by-event-time matrix. Training censoring estimates use out-of-bag trees; new-data estimates use all trees. The names `initial` for new-data initial distributions and `censoring.oob` for training censoring estimates are retained as aliases.

`jfforest.vimp` *Variable importance for a fitted forest*

Description

Variable importance for a fitted forest

Usage

```
jfforest.vimp(
  forest_list,
  feature = NULL,
  seed = NULL,
  method = "permute",
  loss = "default"
)
```

Arguments

`forest_list`	A fitted forest object from `jfforest()`.
`feature`	Name of one fitted feature, or `NULL` (the default) to compute importance for every feature.
`seed`	Random seed for the perturbations. `NULL` draws a seed using R's current random-number state; supply a seed for reproducible results.
`method`	Importance method. `"permute"` (the default) shuffles the selected feature among each tree's OOB observations. `"random"` sends observations to a random daughter whenever that feature is used for a split, with probabilities proportional to the daughter node sizes.
`loss`	Loss function for importance: `"mse"` for regression, `"misc"` or `"brier"` for classification, and `"concordance"`, `"brier"`, or `"kl"` for survival; and `"brier"`, `"kl"`, or `"spherical"` for multi-state forests. `"default"` (the default) selects `"mse"`, `"misc"`, `"concordance"`, or `"brier"` for regression, classification, survival, or multi-state forests, respectively.

Details

Importance compares each tree's OOB loss before and after perturbing the selected feature, without refitting the tree. Positive values mean that perturbing the feature worsens predictions; values near zero indicate little change, and negative values mean that perturbation improves predictions on the OOB observations. Values are signed loss differences, not percentages, and depend on the selected loss and method.

Tree contributions are averaged over trees with a defined OOB loss. For survival with `method = "random"` and `loss = "concordance"`, the average is weighted by each tree's OOB observation count. Concordance importance uses the decrease in C-index, equivalently the increase in `1 - C-index`.

Regression uses mean squared error, and classification uses misclassification rate or the normalised Brier scale described in `jfforest.error()`. Survival and multi-state probability scores use censoring weights and time-normalised integrated losses. Multi-state scores evaluate state occupation probabilities derived from leaf Nelson-Aalen estimators and initial distributions; their state weights are always `1 / num_states` for Brier and spherical scores and 1 for Kullback-Leibler loss, regardless of `state_weights` used during fitting. Censoring weights from the unperturbed leaf assignments are retained when evaluating perturbed predictions.

The forest must have OOB observations; sampling without replacement at `sample_rate = 1` leaves none. Survival concordance also needs comparable survival pairs, and probability scores need observed event times. Required leaf estimates are computed when needed, so fitting with `save_predictions = FALSE` does not prevent VIMP calculation. For an all-feature call, separate seeds are generated for each feature; a single-feature call with the same seed need not give the identical value.

Value

A VIMP-only list with class `jf_vimp`, for both single-feature and all-feature calls. Its `vimp` field is a named list of numeric importance values. Use `result$vimp` for all computed values or `result$vimp[["feature_name"]]` for one feature. Printing displays the first 20 values; use `print(result, n = Inf)` to display all values.

---

jftree *Fit a JumpForests tree*

Description

Fits a single regression, classification, survival, or multi-state tree.

Usage

```
jftree(
  formula,
  data,
  feature_data = NULL,
  splitrule = NULL,
  mtry = NULL,
  min_node_size = NULL,
```

```
  max_depth = NULL,
  nsplits = 10,
  honest = FALSE,
  seed = NULL,
  num_event_times = 0,
  state_weights = NULL,
  fh_weights_a = NULL,
  fh_weights_b = NULL,
  splitrule_par = NULL
)
```

Arguments

formula	Model formula: `y ~ x1 + x2` for regression or classification, `Surv(time, status) ~ x1 + x2` for survival (`status = 1` for an event, `0` for censoring), or `MM ~ x1 + x2` for multi-state models. A numeric response selects regression; a factor, character, or logical response selects classification. Convert numeric class labels to a factor to fit a classification tree. Numeric features are treated as continuous; factor, character, and logical features are treated as categorical.
data	Training data. A data.frame including both responses and features for regression, classification and survival. For multi-state models, a list of lists (one for each observation), each containing two vectors, `"times"` (observed jump times) and `"states"` (the state occupied at the corresponding time). The states must be encoded as integers. Repetition of the final state indicates censoring.
feature_data	Feature data as a data.frame. Only required for multi-state models.
splitrule	Splitting rule. `NULL` uses the model-specific defaults below. For regression, choose `"mse"` (default), `"variance"` (an alias for `"mse"`), `"mae"`, `"binomial"`, `"negativebinomial"`, `"poisson"`, `"gamma"`, `"inversegaussian"`, `"tweedie"`, or `"huber"`. Binomial responses must lie in `[0, 1]`; negative-binomial and Poisson responses must be non-negative integers; Gamma and inverse-Gaussian responses must be positive. Tweedie support follows its power: real for $\xi \leq 0$, non-negative counts at $\xi = 1$, non-negative for $1 < \xi < 2$, and positive for $\xi \geq 2$. For classification, choose `"gini"` (default), `"entropy"`, `"misc"`, `"twoing"` or `"hellinger"`. For survival, choose `"logrank"` (default), `"approxlogrank"` or `"conserve"`. For multi-state models, choose `"logrank"` (default), `"gehan"`, `"taroneware"`, `"approxlogrank"`, `"conserve"`, `"petoprentice"` or `"flemingharrington"`. Choosing `"flemingharrington"` allows for specifying the weights, see the options `"fh_weights_a"` and `"fh_weights_b"`.
mtry	Number of randomly selected candidate features at each split. `NULL` uses `ceiling(sqrt(p))`, where `p` is the number of features.
min_node_size	Minimum number of observations required in each daughter node for a split. `NULL` uses 5 for regression, 1 for classification, 15 for survival, and 20 for multi-state models.
max_depth	Maximum depth of the tree, where the root has depth zero. `NULL` (the default) imposes no depth limit.
nsplits	Number of randomly sampled candidate thresholds per continuous feature, or candidate partitions per categorical feature. Defaults to 10; fewer candidates are

used if fewer distinct splits exist.

`honest` Whether to split the observations into approximately equal sets for choosing splits and estimating leaf predictions. Defaults to `FALSE`, which uses the same observations for both tasks.

`seed` Random seed. `NULL` draws a seed using R's current random-number state; supply a seed for reproducible fitting.

`num_event_times` Maximum number of times in the prediction grid for survival and multi-state models. The default, 0, retains the full grid. A positive value coarsens the grid when necessary; the resulting times are stored in `unique.event.times`.

`state_weights` Numeric vector of finite non-negative weights, one per state in the order `1, ..., num_states`, for multi-state error calculations. Supplied weights are used directly for all scores and are not normalised. `NULL` uses `1 / num_states` per state for Brier and spherical scores and 1 per state for Kullback-Leibler loss. These weights affect error calculations, not splitting or prediction.

`fh_weights_a` Optional finite non-negative Fleming–Harrington a-exponents, one for each state `1, ..., num_states`. `NULL` uses one for every state. Only valid for multi-state trees with `splitrule = "flemingharrington"`. A transition from state `s` is weighted by $p_s(t)^{a_s}(1-p_s(t))^{b_s}$, where $p_s(t)$ is the estimated occupation probability of state `s` in the parent node.

`fh_weights_b` Optional finite non-negative Fleming–Harrington b-exponents, with the same ordering and default as `fh_weights_a`.

`splitrule_par` Numeric scalar for parameterised regression rules: negative-binomial size $k > 0$ (default 1), Tweedie power $\xi \leq 0$ or $\xi \geq 1$, or Huber threshold $\delta > 0$. It is required for Tweedie and Huber and must be `NULL` for every other rule and model type. The default is `NULL`.

Value

A fitted tree object as a list with class `jf_tree`. Use `print()` for a summary and `predict()` for predictions. Multi-state prediction lists print compact summaries. Training predictions are stored in `predictions`. For survival and multi-state trees, `censoring` contains leaf Kaplan-Meier estimates of the censoring survival probability, with one row per observation on `unique.event.times`. The estimates on the full censoring-time grid are in `censoring.full`, with times in `censoring.times`.

`jftree.error` *Compute tree prediction error*

Description

Compute tree prediction error

Usage

```
jftree.error(
  tree_list,
  new_data = NULL,
  jump_data = NULL,
  state_weights = NULL
)
```

Arguments

tree_list	A fitted tree object from jftree().
new_data	Optional evaluation data.frame. For regression, classification, and survival, include all fitted feature and response columns. For multi-state models, include the features here and supply responses in jump_data. Variable types and categorical levels must match the training data; responses must be fully observed apart from the model's censoring representation. NULL returns stored training errors.
jump_data	Multi-state response paths in the same format as jftree()'s data, with one path per row of new_data in the same order. Required only for evaluating multi-state models on new data; defaults to NULL.
state_weights	Multi-state error weights as described in jftree(). With new data, NULL uses the loss-specific defaults, regardless of weights used during fitting. Without new data, stored errors are returned and this argument is ignored.

Details

Without new_data, errors are the stored training errors. With new_data, predictions and errors are computed on those observations. Survival and multi-state probability scores account for censoring using inverse censoring probability weights estimated by the fitted tree.

Value

A named list of error metrics. Smaller losses are better; larger R2 is better.

- Regression: mse.error is mean squared error. R2 is $1 - \mathrm{MSE}/\mathrm{Var}(y)$, using the empirical variance with divisor n on the evaluated responses; it can be negative.
- Classification: BS.error is the squared class-probability error averaged over observations and classes. normalised.BS.error divides this by the score for uniform class probabilities, so 1 is the uniform baseline. misclassification.error.total is the overall proportion of wrong labels; misclassification.error gives this proportion within each observed class (or NA if that class is absent). confusion.matrix contains counts, with observed classes in rows and predicted classes in columns.
- Survival: C.error is 1 - C-index, so smaller values mean better concordance. IBS.error is the integrated Brier score for survival probabilities; IKL.error is the integrated Kullback-Leibler (logarithmic) loss. Their time-normalised counterparts are normalised.IBS.error and normalised.IKL.error.

- Multi-state: `IBS.error`, `IKL.error`, and `IS.error` are the integrated Brier, Kullback-Leibler, and spherical losses for state occupation probabilities, with state weights as specified above. The corresponding fields are `normalised.IBS.error`, `normalised.IKL.error`, and `normalised.IS.error`.

Integrated scores use trapezoidal integration on the fitted event-time grid. Their `normalised.*` versions divide by the grid's final time, giving an average loss over time; normalisation does not guarantee a value between zero and one. This differs from the uniform-probability normalisation used for classification Brier scores.

`jftree.predict` *Predict from a fitted tree*

Description

Predict from a fitted tree

Usage

```
jftree.predict(
  tree_list,
  new_data = NULL,
  compute_censoring = FALSE,
  compute_initial = FALSE
)
```

Arguments

`tree_list`	A fitted tree object from `jftree()`.
`new_data`	Optional data.frame containing all fitted feature names, with the same variable types as the training data. Response columns are not needed; extra columns are ignored. Non-missing categorical values must occur in the training levels. Missing feature values are sent to the right daughter at each split on that feature. `NULL` returns training predictions.
`compute_censoring`	Whether to return censoring survival probabilities estimated by leaf Kaplan-Meier estimators. Defaults to `FALSE`; only valid for survival and multi-state trees. For survival trees this flag applies when `new_data` is supplied; for multi-state trees it also applies to training predictions.
`compute_initial`	Should predicted initial distributions also be returned? Defaults to `FALSE`; only valid for multi-state trees.

Value

Regression: A numeric vector with one predicted response per training observation. With `new_data`, a numeric matrix with one column and one row per observation.

Classification: A numeric vector of predicted class codes for training observations. With `new_data`, a numeric matrix whose first column, `prediction`, contains the class codes and whose remaining columns contain class probabilities, named by `tree_list$class.levels`. Class codes index these class levels. Training class probabilities are stored separately in `tree_list$predictions.prob`.

Survival: A matrix of estimated cumulative hazards (Nelson-Aalen estimators), with one row per observation and one column per time in `tree_list$unique.event.times`. With `new_data` and `compute_censoring = TRUE`, a list containing this matrix in `predictions` and censoring Kaplan-Meier estimates on the same grid in `censoring`. The list also contains `censoring.full`, a matrix of censoring estimates on the censoring-time grid given by `censoring.times`. Without `new_data`, the returned object is the cumulative hazard matrix; stored training censoring estimates can be accessed through `tree_list$censoring` and `tree_list$censoring.full`.

Multi-state: Predictions are a list of individuals when both optional estimates are disabled; access one individual with `result[[i]]`. Otherwise, `result$predictions` contains the Nelson-Aalen estimators. The fields `init` and `censoring` contain the optional estimates when requested. Censoring estimates are Kaplan-Meier censoring survival probabilities, with one row per individual and one column per time in `tree_list$unique.event.times`. When requested, `censoring.full` and `censoring.times` also provide the full censoring-time grid. Multi-state results print compact summaries.

---

occupation_prob *Compute occupation probabilities from initial distribution and Nelson-Aalen estimator*

Description

Computes state occupation probabilities by combining the initial state distribution with the Aalen-Johansen estimator derived from the Nelson-Aalen estimator.

Usage

```
occupation_prob(init, na)
```

Arguments

`init`	A finite numeric vector of initial state probabilities, one per state. Probabilities must be between zero and one and sum to one.
`na`	The Nelson-Aalen estimator for one individual: a non-empty list of equally sized square numeric cumulative intensity matrices in time order. The first matrix must be zero. Diagonal entries must equal the negative sum of the off-diagonal entries in their row.

Details

Pass ‘predictions[[i]]‘ when both optional prediction flags are disabled, or ‘result$predictions[[i]]‘ otherwise. With ‘compute_initial = TRUE‘, use ‘result$init[[i]]‘ for ‘init‘. Invalid inputs or intensity increments that cannot produce valid probabilities raise an informative error.

Value

A list of occupation probability matrices.

---

predict.jf_forest *Predict from a fitted JumpForests forest*

Description

Predict from a fitted JumpForests forest

Usage

```
## S3 method for class 'jf_forest'
predict(object, newdata = NULL, ...)

## S3 method for class 'jf_multistate_forest'
predict(object, newdata = NULL, ...)
```

Arguments

`object`	A fitted forest returned by `jfforest()`.
`newdata`	Optional data frame for prediction. If omitted or `NULL`, return ordinary training-data predictions using all trees. Feature names, types, and categorical levels must meet the `new_data` requirements in `jfforest.predict()`; responses are not required.
`...`	Arguments passed to `jfforest.predict()`, including `compute_initial` and `compute_censoring`. The existing `new_data` spelling is also accepted when `newdata` is omitted.

Value

Regression: A numeric vector with one predicted response per training observation. With `new_data`, a numeric matrix with one column and one row per observation.

Classification: A numeric vector of predicted class codes for training observations. With `new_data`, a numeric matrix whose first column, `prediction`, contains the class codes and whose remaining columns contain class probabilities, named by `forest_list$class.levels`. Class codes index these class levels. Training class probabilities are stored separately in `forest_list$predictions.prob`.

Survival: A matrix of estimated cumulative hazards (Nelson-Aalen estimators), with one row per observation and one column per time in `forest_list$unique.event.times`. With `new_data` and

compute_censoring = TRUE, a list containing this matrix in predictions and censoring Kaplan-Meier estimates on the same grid in censoring. Without new_data, the returned object is the cumulative hazard matrix; stored out-of-bag censoring estimates, when available, can be accessed through forest_list$censoring.oob.

Multi-state: Predictions are a list of subjects when both optional estimates are disabled; access one subject with result[[subject]]. Otherwise, result$predictions contains the subject predictions. With compute_initial = TRUE, the result also has init. With compute_censoring = TRUE, it has censoring, a subject-by-event-time matrix. Training censoring estimates use out-of-bag trees; new-data estimates use all trees. The names initial for new-data initial distributions and censoring.oob for training censoring estimates are retained as aliases.

---

predict.jf_tree *Predict from a fitted JumpForests tree*

Description

Predict from a fitted JumpForests tree

Usage

```
## S3 method for class 'jf_tree'
predict(object, newdata = NULL, ...)

## S3 method for class 'jf_multistate_tree'
predict(object, newdata = NULL, ...)
```

Arguments

object	A fitted tree returned by jftree().
newdata	Optional data frame for prediction. If omitted or NULL, return training-data predictions. Feature names, types, and categorical levels must meet the new_data requirements in jftree.predict(); responses are not required.
...	Arguments passed to jftree.predict(), including compute_initial and compute_censoring. The existing new_data spelling is also accepted when newdata is omitted.

Value

Regression: A numeric vector with one predicted response per training observation. With new_data, a numeric matrix with one column and one row per observation.

Classification: A numeric vector of predicted class codes for training observations. With new_data, a numeric matrix whose first column, prediction, contains the class codes and whose remaining columns contain class probabilities, named by tree_list$class.levels. Class codes index these class levels. Training class probabilities are stored separately in tree_list$predictions.prob.

Survival: A matrix of estimated cumulative hazards (Nelson-Aalen estimators), with one row per observation and one column per time in tree_list$unique.event.times. With new_data and

compute_censoring = TRUE, a list containing this matrix in predictions and censoring Kaplan-Meier estimates on the same grid in censoring. The list also contains censoring.full, a matrix of censoring estimates on the censoring-time grid given by censoring.times. Without new_data, the returned object is the cumulative hazard matrix; stored training censoring estimates can be accessed through tree_list$censoring and tree_list$censoring.full.

Multi-state: Predictions are a list of individuals when both optional estimates are disabled; access one individual with result[[i]]. Otherwise, result$predictions contains the Nelson-Aalen estimators. The fields init and censoring contain the optional estimates when requested. Censoring estimates are Kaplan-Meier censoring survival probabilities, with one row per individual and one column per time in tree_list$unique.event.times. When requested, censoring.full and censoring.times also provide the full censoring-time grid. Multi-state results print compact summaries.

`print.jf_forest` *Print a fitted JumpForests forest*

Description

Print a fitted JumpForests forest

Usage

```
## S3 method for class 'jf_forest'
print(x, ...)

## S3 method for class 'jf_multistate_forest'
print(x, ...)
```

Arguments

x	A fitted forest returned by [jfforest()].
...	Additional arguments, currently unused.

Value

The fitted forest, invisibly. Prints the summary from [print_forest()].

print.jf_tree *Print a fitted JumpForests tree*

Description

Print a fitted JumpForests tree

Usage

```
## S3 method for class 'jf_tree'
print(x, full = FALSE, ...)

## S3 method for class 'jf_multistate_tree'
print(x, full = FALSE, ...)
```

Arguments

x	A fitted tree returned by [jftree()].
full	If 'TRUE', also print the full tree structure.
...	Additional arguments, currently unused.

Value

The fitted tree, invisibly. Prints the summary from [print_tree()].

print.jf_vimp *Print forest variable importance*

Description

Print forest variable importance

Usage

```
## S3 method for class 'jf_vimp'
print(x, n = 20L, ...)
```

Arguments

x	A VIMP result returned by [jfforest.vimp()].
n	Maximum number of feature values to display. Defaults to 20. Use 'Inf' to display all computed values.
...	Additional arguments passed to [print.data.frame()].

Value

The VIMP result, invisibly. Prints a table of importance values and instructions for accessing them.

print_forest *Print information about a fitted JumpForests forest*

Description

Displays summary information about a fitted random forest, including forest type, data info, and out-of-bag errors (only available if predictions are computed and saved in the forest, i.e. if save_predictions = 'TRUE')

Usage

```
print_forest(forest_list)
```

Arguments

forest_list	A fitted forest object returned by jfforest.

Value

Invisible NULL. Prints information to console as a side effect.

print_tree *Print information about a fitted JumpForests tree*

Description

Displays summary information about a fitted tree, including tree type, data info, and training/prediction errors. Can optionally print full tree details.

Usage

```
print_tree(tree_list, full = FALSE)
```

Arguments

tree_list	A fitted tree object returned by jftree.
full	If 'TRUE', also prints the full tree structure with all split thresholds and feature IDs. Default is 'FALSE'.

Value

Invisible NULL. Prints information to console as a side effect.

`transition_prob` *Compute transition probabilities via the Aalen-Johansen estimator from a Nelson-Aalen estimator by directly applying the product integral. Only valid for Markov models.*

Description

Compute transition probabilities via the Aalen-Johansen estimator from a Nelson-Aalen estimator by directly applying the product integral. Only valid for Markov models.

Usage

```
transition_prob(na)
```

Arguments

`na`	The Nelson-Aalen estimator for one individual: a non-empty list of equally sized square numeric cumulative intensity matrices in time order. The first matrix must be zero. Diagonal entries must equal the negative sum of the off-diagonal entries in their row.

Details

Pass ‘predictions[[i]]‘ when both optional prediction flags are disabled, or ‘result$predictions[[i]]‘ otherwise. Invalid inputs or intensity increments that cannot produce valid probabilities raise an informative error.

Value

A list of Aalen-Johansen estimator matrices.

Index

jfforest, 2, *19*
jfforest(), *5*, *7, 8*, *15*
jfforest.error, 5
jfforest.error(), *9*
jfforest.predict, 6
jfforest.predict(), *15*
jfforest.vimp, 8
jftree, 9, *19*
jftree(), *12, 13*, *16*
jftree.error, 11
jftree.predict, 13
jftree.predict(), *16*
JumpForests *(*JumpForests-package*)*, 2
JumpForests-package, 2

occupation_prob, 14

predict.jf_forest, 15
predict.jf_multistate_forest
        *(*predict.jf_forest*)*, 15
predict.jf_multistate_tree
        *(*predict.jf_tree*)*, 16
predict.jf_tree, 16
print.jf_forest, 17
print.jf_multistate_forest
        *(*print.jf_forest*)*, 17
print.jf_multistate_tree
        *(*print.jf_tree*)*, 18
print.jf_tree, 18
print.jf_vimp, 18
print_forest, 19
print_tree, 19

transition_prob, 20